

(1) addl (%eax), %edx

edx: 0x00000070

eax: 0x08049380, 0xfffffff0

执行操作: $0x08049380 + 0x00000070 = 0x080493f0$
 $= 0x00000070$

EDX 变为 0x00000070

CF=1, OF=0, ZF=0, SF=0

(2) subl (%eax, %ebx), %ecx

地址 = EAX + EBX = 0x08049400, 内存为 0x80000008

运算: $0x00000010 - 0x80000008 = 0x80000008$

CF=1, OF=1, ZF=0, SF=1 ECX 变为 0x80000008

(3) orw 4(%eax, %ecx, 8), %bx

地址 = EAX + ECX * 8 + 4 = 0x08049384

运算: $0x0100 | 0xffff00 = 0xffff00$

EBX 整体变为 0x0000ffff

CF=0, OF=0, ZF=0, SF=1

(4) testb \$0x80, %dl

运算: $0x80 \& 0x80 = 0x80$

不改变内容

CF=0, OF=0, ZF=0, SF=1

(1)

decw %cx

运算: $0x0010 - 1 = 0x000f$

ECX 变为 0x0000000f

CF=不变

OF=0

ZF=0

SF=0

(5) imull \$32, (%eax, %edx), %ecx

地址 = EAX + EDX = 0x08049380, 内存为 0x908f12a8

运算: $32 \times 0x908f12a8 = 0x11e25500$

ECX 变为 0x11e25500

CF=1, OF=1

(6) mulw %bx

AX=0x300, BX=0x0100

DX=AX = AX * BX = 0x00930000

EAX 变为 0x08049380, EDX 变为 0x00000093

CF=1, OF=1



二

(1) 1. `movb 8(%ebp), %dl`

从栈中取得第一参数 x , 存入 `%dl` 寄存器。

2. `movl 12(%ebp), %eax`

从栈中取得第二参数 x , 存入 `%eax` 寄存器。

3. `testl %eax, %eax`

检查寄存器 `%eax` (即指针 p) 是否为 0。

4. ~~`je .L1`~~

`je .L1`
如果 p 是空指针, 跳转到 `.L1` 结束。

5. `testb $0x80, %dl`

检查 `%dl` (即 x) 的最高位是否为 1 (x 为负)

6. `je .L1`

如果结果为 0 ($x \geq 0$) 跳转到 `.L1`

7. `addb %dl, (%eax)`

将 `%dl` 中的值加到 `%eax` 指向的内存地址中

8. `.L1:`

跳转标记

因为 `if` 具有“短路求值”特性, 如果 p 为假, 必须跳过后续的 $x < 0$ 判断。

7. `void comp(char x, int *p) {`

`bool c1 = (p == 0);`

`if (c1) goto done;`

`bool c2 = ( $x < 0$   $x >= 0$ );`

`if (c2) goto done;`

`*p += x;`

`done:`
`return;`

`}`



内存地址	存储内容	含义
0xbffco7ec	C8 85 04 08	返回地址.
0xbffco7e8	00 08 fc bf	EBP 旧值
0xbffco7e4	08 00 00 00	EDI 旧值
0xbffco7e0	10 00 00 00	ESI 旧值
0xbffco7dc	05 00 00 00	EBX 旧值
0xbffco7d8	随机	局部变量空间
0xbffco7d4	随机	buf 缓冲区溢出
...	...	...
0xbffco7c0	空闲	当前 %esp 指向的位置

...	7ec	"8910" "8" "9" "\0" 08	38 39 00 08
...	7e8	"4567"	34 35 36 37
...	7e4	"0123"	30 31 32 33
...	7e0	"CDEF"	43 44 45 46
...	7dc	"89AB"	38 39 41 42
...	7d8	"4567"	34 35 36 37
...	7d4	"0123"	30 31 32 33

(3) 正确返回地址: 0X080485C8

错误的返回地址: 0X08003938

(4) . %ebx, %esi, %edi, %ebp, %eip

(5) ① 内存分配大小错误:

strlen(buf) 不包含末尾的 '\0', 导致堆溢出

② 未检查 malloc 的返回值

